

Structure from Motion Alignment with Graph Neural Networks

Jack Daus

jdaus2@gmu.edu

George Mason University

December 9, 2025

1 Introduction

1.1 Structure from Motion

In computer vision, Structure from Motion (SfM) is the task of inferring 3D structure from multiple 2D images. A common SfM pipeline is composed of three sequential steps: (1) *feature extraction*, where each image is processed independently to extract distinctive *keypoints*; (2) *feature matching*, where correspondences are established between images that view common scene elements; and (3) *bundle adjustment*, which incrementally registers camera poses and reconstructs a sparse 3D point cloud. We refer to the output of this pipeline as a “SfM model,” containing the registered camera poses and 3D points.

1.2 Problem Statement

Sometimes the SfM pipeline fails to register all images into a single model. When bundle adjustment cannot find a next image to add, it starts a new, separate model with the remaining images. This results in two or more disconnected SfM models that share no common cameras. Merging these models can sometimes require manual intervention.

1.3 Motivation

One particularly challenging scenario for SfM pipelines is scenes with repeated patterns [1]. Images viewing repeated patterns risk being incorrectly registered to each other. To disambiguate repeated structures, it can help to use “global reasoning” that leverages information beyond the local neighborhood [2].

Recent work has demonstrated the effectiveness of graph neural networks for various SfM sub-problems [3], [4], [5]. While these approaches focus on camera pose estimation from scratch, we address a complementary problem: merging disconnected SfM models that were produced by a traditional pipeline. We propose to explore whether a graph neural network can learn both local and global features to enable correct registration in these difficult scenarios.

2 Problem Definition

We frame this problem as a graph-level attribute regression problem. Given two distinct SfM graphs, the task is to predict a rigid transformation that aligns one graph’s reference frame to the other. Below, we formally define how we cast SfM models as graphs and specify the learning task.

2.1 Graph Definition

We cast a SfM model as a graph. To do this, we consider each camera, c_i , in the SfM model to be a node, $c_i \in V$. We define an edge between two nodes if and only if the cameras see at least one common image keypoint. That is, if camera i and camera j see the same keypoint k , then $\{c_i, c_j\} \in E$.

$$G = (V, E) \quad (1)$$

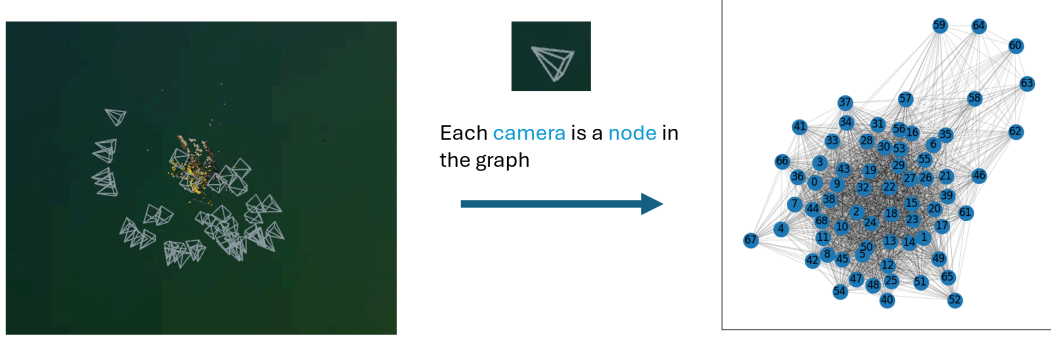


Figure 1: Casting a SfM model as a graph. Each camera becomes a node, and edges connect cameras that share at least one common keypoint.

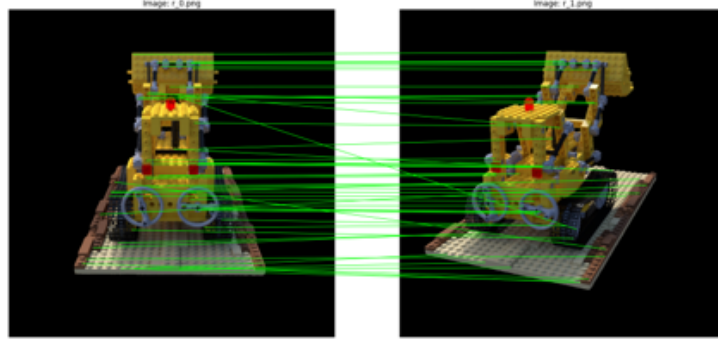


Figure 2: Illustration of keypoint matches between two images. Matching keypoints establish edges in the SfM graph.

2.2 Node Features

We define our node features with information derived from the SfM model. Each node has features for position, rotation, and the RGB image. The position, $\mathbf{p}$, defines the coordinates of the camera in space, (x, y, z) . The rotation, $\mathbf{q}$, is a quaternion that defines the orientation of the camera in space. The image data, $\mathbf{I}$, is an RGB image of dimensions L pixels by W pixels.

$$c_i = (\mathbf{p}_i, \mathbf{q}_i, \mathbf{I}_i) \quad (2)$$

$$\mathbf{p}_i \in \mathbb{R}^3 \quad (3)$$

$$\mathbf{q}_i \in \mathbb{R}^4, \quad \|\mathbf{q}_i\| = 1 \quad (4)$$

$$\mathbf{I}_i \in \mathbb{R}^{L \times W \times 3} \quad (5)$$

2.3 Learning Task

We consider two separate SfM models, SfM_a and SfM_b , whose cameras form disjoint sets:

$$V_a \cap V_b = \emptyset \quad (6)$$

Each SfM model exists in its own arbitrary reference frame. If we translate or rotate all cameras in a model, we have not meaningfully changed the scene—only its coordinate representation. When two models capture the same scene, we may wish to align them to a common reference frame so that camera positions correspond correctly. A reference frame, F_i , is defined as a rigid transform, $F_i \in \text{SE}(3)$. Where a rigid transform is defined as any rotation and translation pair:

$$\text{SE}(3) = \{(\mathbf{R}, \mathbf{t}) \mid \mathbf{R} \in \text{SO}(3), \mathbf{t} \in \mathbb{R}^3\} \quad (7)$$

Our goal is to find a rigid transformation, $T_{ab} \in \text{SE}(3)$, that aligns the reference frame of G_b with that of G_a . Specifically, we want to learn a rotation $\mathbf{R} \in \text{SO}(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$ such that,

$$F_a = T_{ab} \cdot F_b \quad (8)$$

where F_a and F_b denote the reference frames of G_a and G_b , respectively.

3 Methodology

3.1 Dataset

To construct a dataset, we use the *lego* images from the NeRF dataset [6]. The *lego* dataset consists of 69 photorealistic synthetic images of a lego bulldozer. These images were generated by path tracing. We then created the SfM model by running the standard COLMAP pipeline with default settings [7].

We use a self-supervised approach to learn this task. To generate the data, we take this complete SfM model and then create two disjoint subgraphs, G_a and G_b , by sampling the cameras. We sample 15 cameras for each subgraph. We chose the sample size 15 as reasonable approximation of what a wild SfM model might have that is proportional to the particular 69 image model we are working with. We designed a sampling procedure that does not sample at random. We do not sample at random because we intuitively think this would be too easy for the model to learn. When we sample at random, each subgraph will inevitably have a diverse range of view points across the object. This makes the challenge too easy, since it creates many points of view for the subgraph. Instead, to make the task more challenging and to mimic scenarios encountered in the wild, we cluster the sampled cameras for each subgraph. More specifically, we begin by sampling two random cameras, c_{a1} and c_{a2} . We then iteratively add a camera to each group. At each step, we add the camera that has the minimum distance to any camera in the group. We alternate between the two groups until either 15 cameras have been sampled, or there are no more co-visible cameras available, whichever comes first. Two cameras are *co-visible* if they share at least one common keypoint.

We run this sampling procedure 1,000 times to create 1,000 pairs of subgraphs.

$$\text{Samples} = \{(G_{a_1}, G_{b_1}), (G_{a_2}, G_{b_2}), \dots, (G_{a_{1000}}, G_{b_{1000}})\} \quad (9)$$

At this point, all pairs of subsamples (G_{a_i}, G_{b_i}) still have the same reference frame. That is, $F_{a_i} = F_{b_i}$. In order to force them to have different reference frames, we apply a random perturbation (a random rotation and translation) to the reference frame for G_{b_i} :

$$\tilde{F}_{b_i} := [\mathbf{R}_i \mid \mathbf{t}_i] \cdot F_{b_i} \quad (10)$$

Where we sample from,

$$\mathbf{R}_i \sim \text{Uniform}(\text{SO}(3)) \quad (11)$$

$$\mathbf{t}_i \sim \text{Uniform}([-3, 3]^3) \quad (12)$$

In practice, what it means to apply this perturbation is to apply this rotation and translation uniformly to all node features in the graph. That is, for each node $c_k \in G_b$, we apply the perturbation to the position and rotation,

$$(\tilde{\mathbf{p}}_k, \tilde{\mathbf{q}}_k) := (\mathbf{R}_i, \mathbf{t}_i) \circ (\mathbf{p}_k, \mathbf{q}_k) \quad (13)$$

We define this sampled perturbation $(\mathbf{R}_i, \mathbf{t}_i)$ as the ground truth label for each training pair. The network’s task is to predict this transformation from the two graphs.

For image features, we downsample each image from 800×800 pixels to 5×5 pixels. We then flatten the images into a single vector. This yields a 75-dimensional ($5 \times 5 \times 3$) feature vector that is part of each node, in addition to the position and rotation features.

We split the 1,000 generated sample pairs into training, validation, and test sets using a 70-20-10 split (700 training, 200 validation, 100 test). We train using the Adam optimizer with a step scheduler with a step size of 100 and a gamma of 0.8.

We evaluate performance using two metrics: translation error and angular error:

$$\text{Translation Error} = \|\mathbf{t}_{\text{pred}} - \mathbf{t}_{\text{gt}}\|_2 \quad (14)$$

$$\text{Angular Error} = (2 \cdot \arccos(|\mathbf{q}_{\text{pred}} \cdot \mathbf{q}_{\text{gt}}|)) \cdot \frac{180}{\pi} \quad (15)$$

3.2 Learning Objective

Given any two SfM graphs, the goal is to learn a function f_θ , parameterized by learnable weights θ , that predicts a rigid transformation.

$$f_\theta(G_a, G_b) = (\mathbf{R}, \mathbf{t}) \quad (16)$$

Our function is of the form:

$$f_\theta : \mathcal{G} \times \mathcal{G} \rightarrow \text{SE}(3) \quad (17)$$

where $\mathcal{G}$ denotes the space of all valid SfM graphs. The goal is to register the cameras in G_b correctly with respect to the coordinate space of G_a . We experiment with several loss functions in our experiments below. Here, we discuss the core elements of the loss functions explored.

3.2.1 Translation Loss

We use the mean squared error (MSE) between predicted and ground truth translations:

$$\mathcal{L}_{\mathbf{t}} = \|\mathbf{t}_{\text{pred}} - \mathbf{t}_{\text{gt}}\|_2^2 \quad (18)$$

3.2.2 Rotation Loss

We experiment with multiple rotation representations and corresponding loss functions:

Quaternion representation. We parameterize rotations as unit quaternions $\mathbf{q} \in \mathbb{R}^4$. We use either an MSE loss (L2) or a quaternion dot product loss [8] that accounts for the double-cover property of quaternions [9]:

$$\mathcal{L}_{\mathbf{R}}^{\text{quat}} = 1 - |\mathbf{q}_{\text{pred}} \cdot \mathbf{q}_{\text{gt}}| \quad (19)$$

9D rotation matrix representation. We also experiment with predicting a 9-dimensional vector representing a 3×3 rotation matrix. We calculate the Frobenius norm of the difference between the predicted and ground truth matrices, similar to the approach in [10]. To encourage valid rotation matrices, we optionally add a regularization term based on the Frobenius norm between the predicted matrix and its closest valid $\text{SO}(3)$ matrix:

$$\mathcal{L}_{\mathbf{R}}^{\text{matrix}} = \|\mathbf{M}_{\text{pred}} - \mathbf{R}_{\text{gt}}\|_F + \lambda_{\text{reg}} \|\mathbf{M}_{\text{pred}} - \text{proj}_{\text{SO}(3)}(\mathbf{M}_{\text{pred}})\|_F \quad (20)$$

3.2.3 Combined Loss

When learning both rotation and translation, we combine the losses with a weighting factor λ that modulates the relative importance of the translation loss. The importance of this weighting factor is noted by [11].

$$\mathcal{L} = \mathcal{L}_{\mathbf{R}} + \lambda \cdot \mathcal{L}_{\mathbf{t}} \quad (21)$$

3.3 Model Architecture

We adopt a Siamese graph neural network architecture, where two GNN encoders with shared weights process the input graphs G_a and G_b independently.

Each GNN encoder consists of two graph convolution layers followed by ReLU activations. We experiment with two types of graph convolution: Graph Convolutional Networks (GCN) and Graph Attention Networks (GATv2). After the convolution layers, we apply global mean pooling to aggregate node embeddings into a single graph-level embedding.

The graph embeddings from both branches are concatenated and passed through a shared MLP prediction head that outputs both the rotation and translation predictions. This fused prediction head, rather than separate heads for rotation and translation, has been shown to improve performance when the two quantities are related [11].

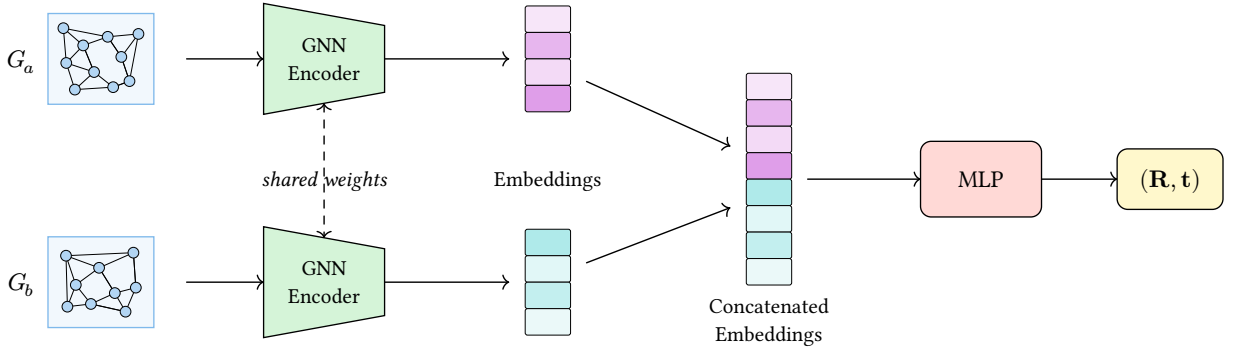


Figure 3: Siamese graph neural network architecture for SfM alignment. Both encoder branches share weights.

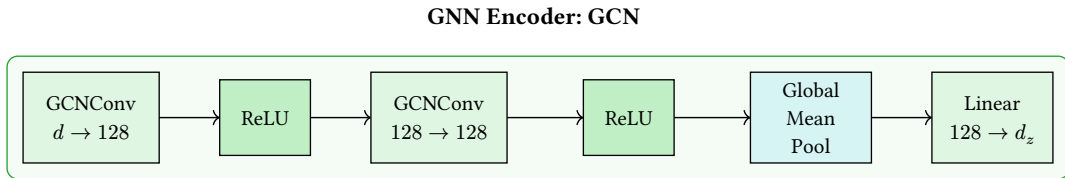


Figure 4: GCN encoder architecture. d is the input node feature dimension, d_z is the graph embedding dimension.

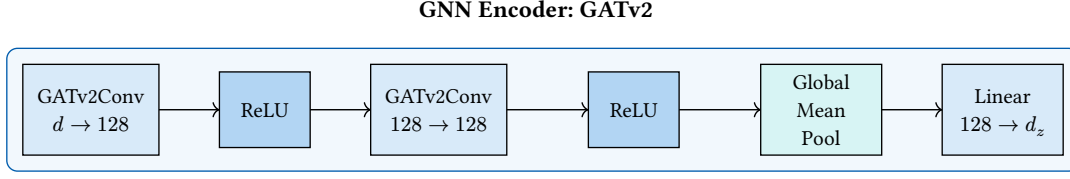


Figure 5: GAT encoder architecture using GATv2Conv layers. d is the input node feature dimension, d_z is the graph embedding dimension.

4 Experiments

In the following sections, we explore four experiments:

- Learning translation only
- Learning rotation only
- Learning both translation and rotation
- Using images in node features

4.1 Experiment 1: Learning Translation Only

In this first experiment, we aim to learn only a translation perturbation. We do not perturb the data with any rotation. In this simplified setting, we test whether the network can learn this simpler task. As described in the Dataset section above, we create a dataset of 1,000 disjoint subgraph pairs. We perturb each G_{b_i} by some uniformly sampled value $\mathbf{t}_i \sim \text{Uniform}([-3, 3]^3)$. The network then aims to predict this translation. We use MSE for the loss function.

For the perturbation defined in equation Equation 13, we simply set $\mathbf{R}_i$ to the identity matrix (i.e., no rotation).

For all experiments, we use batch size of 64, use a learning rate of 0.001. We train for 1,000 epochs with a scheduler with step size of 200 and gamma of 0.8. We picked these settings based on informal experimentation.

In this experiment, we vary the following parameters:

- embedding size in $\{64, 256\}$
- embedding layer type: GCN or GATv2

In the results, we found that the GATv2-based network performed slightly better than the GCN-based network on translation error on the test dataset (0.077 compared to 0.113). We also compared embedding sizes 64 and 256 for the GAT based architecture. Here, there was a small but negligible difference in the translation error on the test dataset (less than 0.003 difference).

Experiment	Embedding Layer	Embedding Size	Image	Train	Val	Test
1.0	GCN	64	False	0.0468	0.1038	0.1135
1.1	GATv2	64	False	0.0370	0.0794	0.0768
1.2	GATv2	256	False	0.0284	0.0934	0.0744

Table 1: Results for Experiment 1 - Translation Only

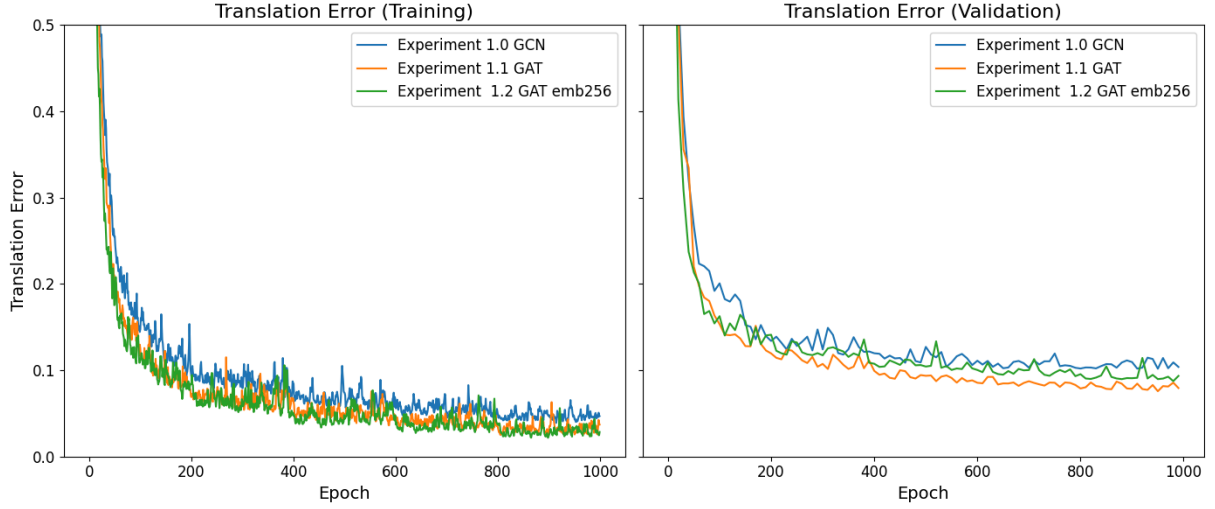


Figure 6: Training curves for Experiment 1 (translation only). The GATv2 architecture achieves lower translation error than GCN.

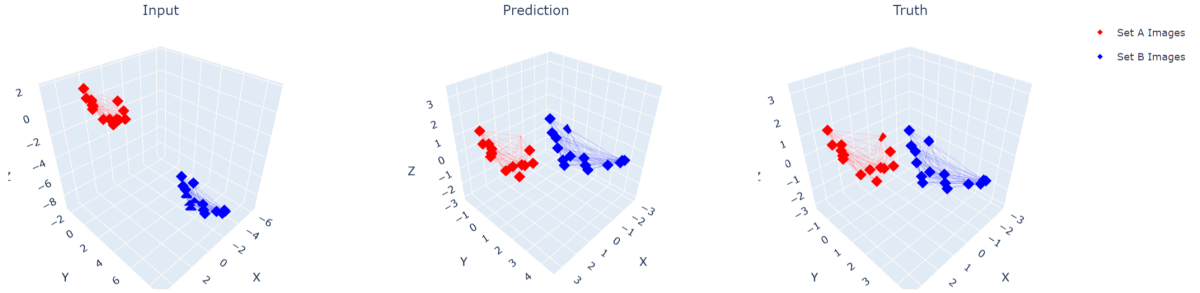


Figure 7: Visualization of predicted graph alignments. Each subplot shows G_a (red) and G_b (blue).

4.2 Experiment 2: Learning Rotations Only

In this next set of experiments, we aim to learn only a rotation perturbation. Similar to above, isolating this aspect helps us better understand the dynamics of rotation learning before adding translation.

Here, we apply a uniformly random rotation to each graph, G_b . That is, we apply this rotation to all the $SE(3)$ attributes of the nodes of the graph. For the perturbation defined in equation Equation 13, we simply set $t_i = \mathbf{0}$.

We run all training runs with adam optimizer for 1,000 epochs with a step learning rate decay of 0.8 decay every 100 steps. We use a batch size of 32 with a GCN-based architecture. (We did not test GAT since the model already fit the training data well, so capacity seemed sufficient for this scenario.)

In this set of experiments, we use the following parameter settings:

- learning rate $\in \{0.01, 0.001\}$
- loss function $\in \{L1, L2, \text{quaternion_loss}\}$
- rotation representation $\in \{\text{quaternion}, 9D \text{ rotation matrix}\}$
- 9D regularizer frobenius norm = True | False

We report the angular error as our metric of interest in this experiment.

We found that in this setting, a learning rate of 0.001 consistently outperformed a learning rate of 0.01.

We explore the choice of rotation representation for our prediction. We find that quaternion-based rotation representation outperforms matrix-based representation. In earlier experiments, we noticed the opposite trend—that matrix representations outperformed quaternion representations. It was only after refactoring the code that this trend reversed. The author is still investigating this discrepancy. The fact that quaternions outperform rotation matrices is unexpected, given previous research [9].

Another unexpected result is that the MSE loss performs best. Initially, we thought this naive approach would be problematic due to the well-known “double cover” issue for quaternion representations.

By comparing results from experiment 2.0.a and 2.1, we see that the added regularization term of the frobenius norm did not improve the angular error.

Overall, the model seems to learn some of the rotation task, but struggles to generalize completely. The training error approaches 0 degrees, but the validation and test error remain relatively high at > 25 degrees of rotation error.

Exp.	Model	Emb Size	LR	Loss	Regularizer	Rot Rep	Image	Train	Val	Test
2.0.a	GCN	64	0.001	L1	frob. norm	matrix	False	1.06	37.41	34.32
2.0.b	GCN	64	0.01	L1	frob. norm	matrix	False	24.38	61.11	58.47
2.1	GCN	64	0.001	L1		matrix	False	1.01	32.88	32.45
2.2	GCN	64	0.001	quat_dot		quat	False	1.42	51.52	46.13
2.3	GCN	64	0.001	L2		quat	False	0.98	25.03	26.92

Table 2: Angular error (degrees) for Experiment 2

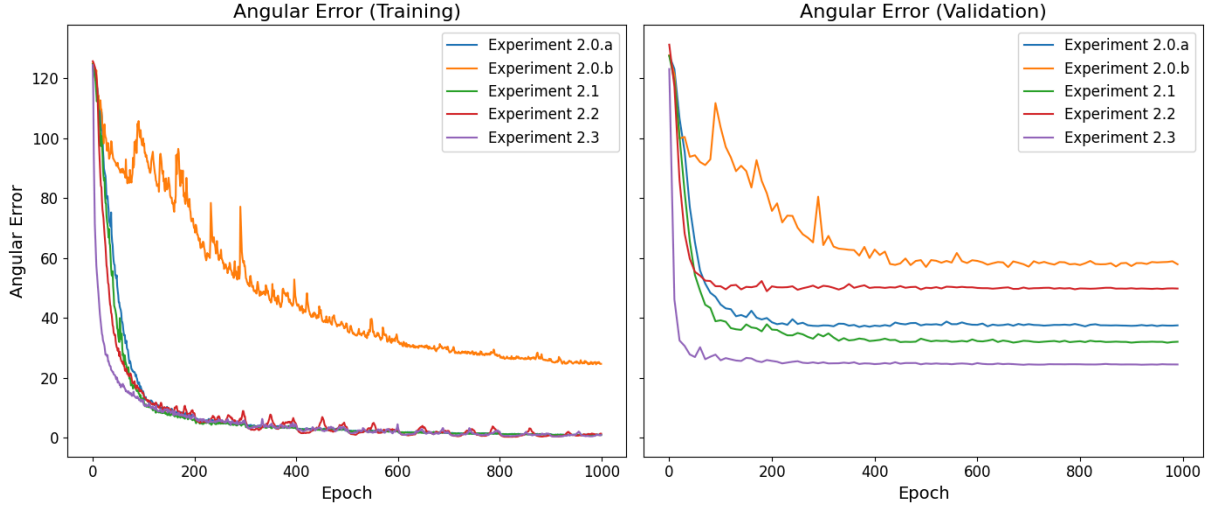


Figure 8: Angular error training curves for Experiment 2. The quaternion representation with L2 loss (Exp 2.3) achieves the lowest test error.

4.3 Experiment 3: Learning Both Translation and Rotation

In this experiment, we aim to learn both a translation and rotation perturbation.

Based on previous experiment results, we use the quaternion representation with an L2 loss for rotation. Here, we test different values of λ for weighting the rotation loss versus the translation loss. This weighting is an important factor when the prediction heads are fused [11].

Again, we train for 1000 epochs using a batch size of 32, and a learning rate of 0.001.

We test two different λ values: $\{1.0, 0.1\}$

Here, we observe that the network is able to learn both translation and rotation together. However, when we compare to models that learn only one perturbation, these results are much worse. Regarding translation error on the test dataset, experiment 3.1 has much greater error compared to experiment 1.0, where we learned only translation (1.48 compared to 0.11, respectively). Regarding rotation, we find something similar: experiment 3.1 has much higher test angular error compared to experiment 2.3, where only rotation was predicted (67.5 degrees versus 26.9 degrees, respectively).

In addition, the error plots show signs of overfitting: the training error approaches 0, but the validation error increases after hitting a minimum early in the training regime.

Exp.	Model	Emb Size	LR	Loss	Rot Rep	λ Trans	Image	Angular Error			Translation Error		
								Train	Val	Test	Train	Val	Test
3.0	GCN	64	0.001	mse	quat	1.0	False	8.33	68.66	73.56	0.05	1.68	1.49
3.1	GCN	64	0.001	mse	quat	0.1	False	2.28	68.76	67.51	0.06	1.68	1.55

Table 3: Results for Experiment 3. Angular error in degrees, translation error in L2 distance.

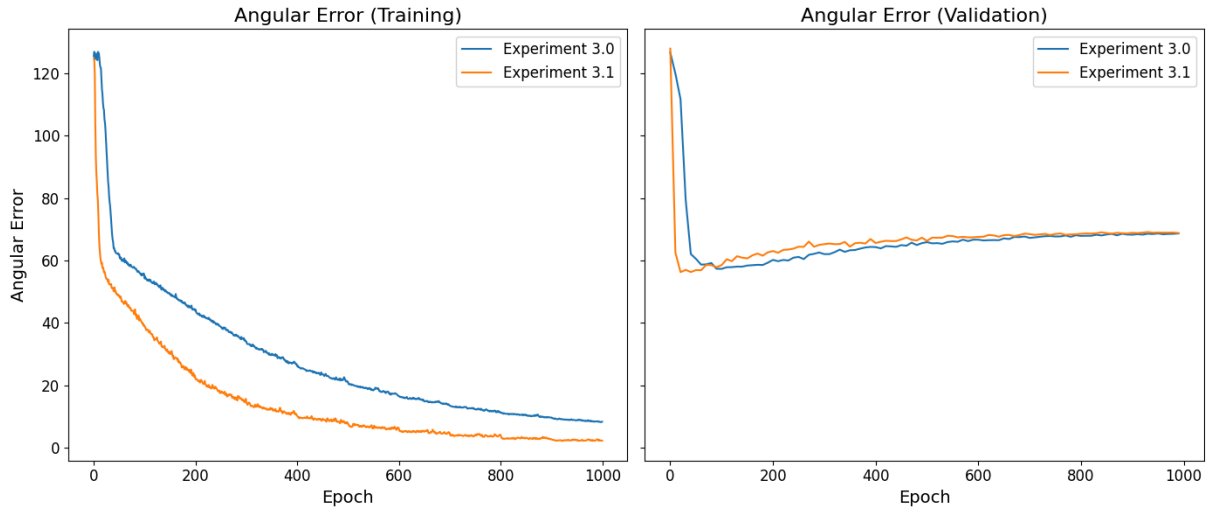


Figure 9: Angular error training curves for Experiment 3. Both configurations show signs of overfitting.

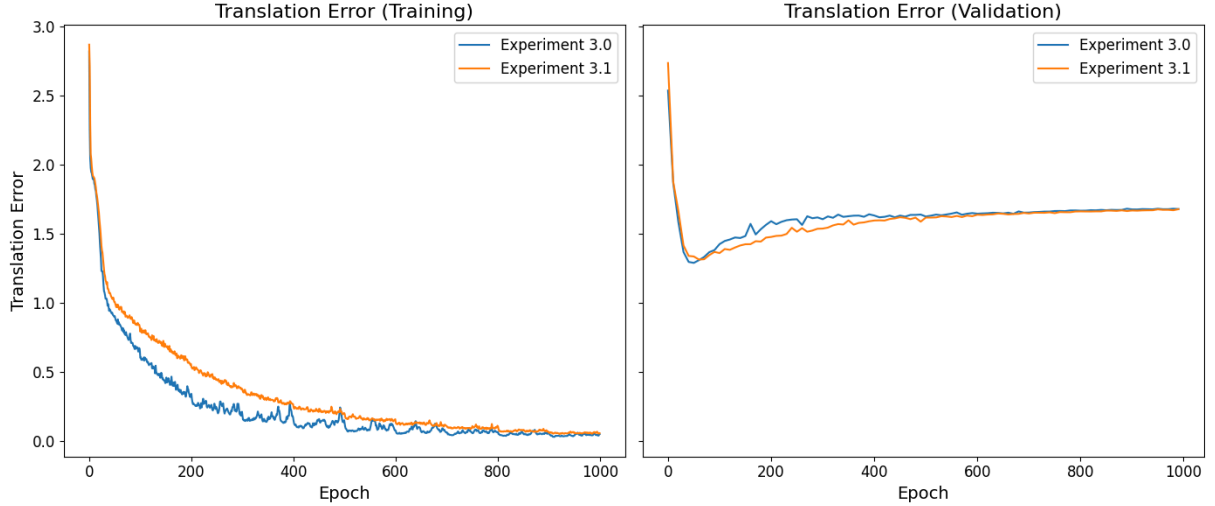


Figure 10: Translation error training curves for Experiment 3. Both configuration show signs of overfitting.

4.4 Experiment 4: Learning with Image Features

In this experiment, we add images to the features for each node. We downsample the images from 800×800 to 5×5 pixels and flatten across all three color channels into a vector, yielding $5 \times 5 \times 3 = 75$ dimensions of image features.

This is aggressive downsampling. We do this to prevent an explosion of feature dimensions that might overwhelm the position (3 dimensions) and rotation (4 dimensions) features for each node.

(In an earlier experiment with 100×100 downsampled images, this resulted in 30,000 dimensions associated with the image. This negatively impacted the training performance, since these many dimensions caused the position and orientation data to be lost in the noise.)

As we can see from the results, adding image features improves the model performance. We see that the test error is lower for both translation and rotation error. The angular error is remarkably lower (18.81 degrees versus 67.51). The translation error is also notably lower, 1.07 compared to 1.55. In addition, the validation error curve flattens out, rather than showing signs of overfitting as when these image features are absent (as seen in experiment 3.1).

Exp.	Model	Emb Size	LR	Loss	Rot Rep	Image Feat	λ Trans	Angular Error			Translation Error		
								Train	Val	Test	Train	Val	Test
3.1	GCN	64	0.001	mse	quat	False	0.1	2.28	68.76	67.51	0.06	1.68	1.55
4.1	GCN	64	0.001	mse	quat	True	0.1	1.27	15.57	18.81	0.03	1.03	1.07

Table 4: Results for Experiment 4. Including image features leads to a noticeable improvement in test error.

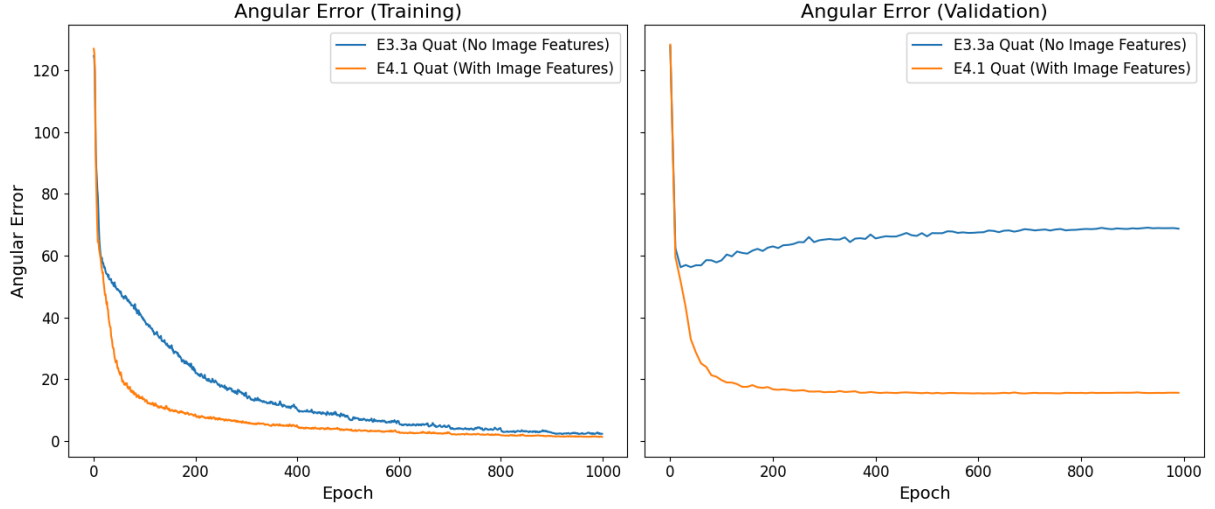


Figure 11: Angular error training curves for Experiment 4. Adding image features reduces overfitting and improves generalization.

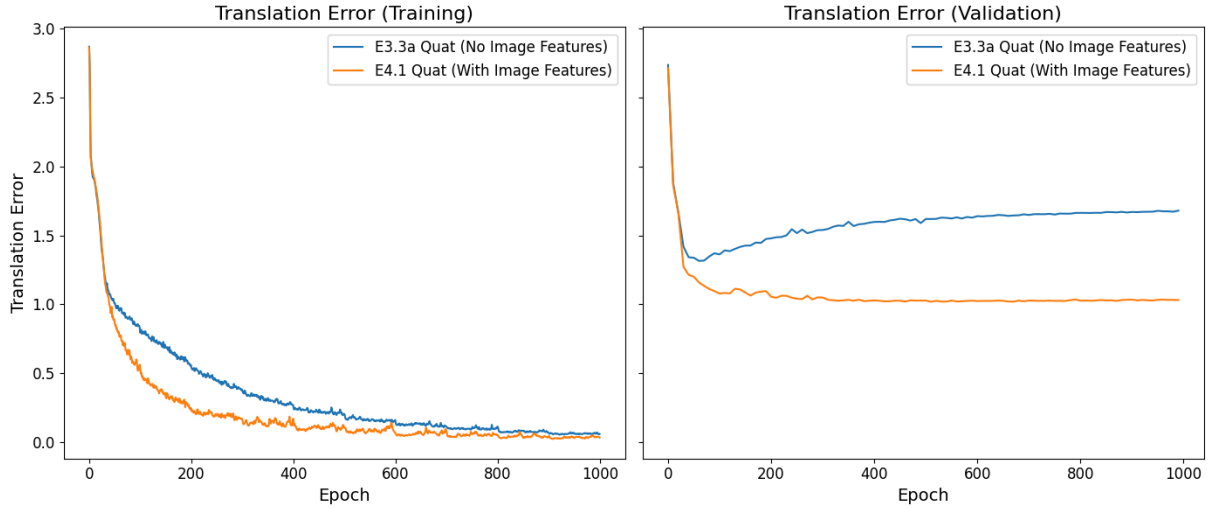


Figure 12: Translation error training curves for Experiment 4. Similar to the curve for angular error, we see reduced overfitting and improved generalization.

5 Future Work

5.1 Limitations

There are several limitations with the current study that could be improved in a follow-on. In particular, in this study, we only train our network on a single SfM model. In order for a truly useful model to be developed, we would want a model that can generalize to previously unseen SfM models. To achieve this, we would need to collect a large set of existing SfM models to add to our self-supervised approach. For example, datasets like MegaScenes [12] could be used.

Another limitation is that although we predict a translation and rotation, we do not predict a scale factor. In a real-world setting, the COLMAP structure from motion system produces a model that is defined up to a scale. Therefore, when we match the frames of references between two SfM models, the scale data is typically not present. We would most likely need to apply some kind of scale transform

to align them. More formally, instead of learning to predict an element in $SE(3)$, we could extend this network to predict an element in $Sim(3)$.

Another limitation is that we do not perturb the node features of our graph G_a in our setup. This may encourage the model to overfit to the specific frame of reference represented in G_a . In particular, when a camera c_i is present in G_a in our self-supervised dataset, this camera will always have the same position and rotation values. This encourages the network to memorize these values. To address this, we should also perturb the position and rotation values of G_a in addition to what we already do for G_b .

In addition, in a future study, we could make better use of the image data present in the SfM graph. In this study, due to computation and time constraints, we heavily downsampled images to 5×5 pixels. We found evidence that even this limited information improves model efficacy. We could do much better by using a CNN to compress the images to a manageable representation size per node. We might also leverage an off-the-shelf visual embedding network like DINOv2. Similar approaches were taken by studies like VGGsFM [13] and VGGT [14].

Finally, there is much more experimentation we can do with model architectures. In particular, experimenting with different ways to create graph embeddings that are invariant to rotation, translation, and scale changes, similar to what was achieved recently in a $Sim(3)$ -equivariant architecture for point cloud prediction [15].

References

- [1] R. Roberts, S. N. Sinha, R. Szeliski, and D. Steedly, “Structure from motion for scenes with large duplicate structures,” in *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Fort Collins, CO, Jun. 2011.
- [2] C. Zach, M. Klopschitz, and M. Pollefeys, “Disambiguating visual relations using loop constraints,” in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2010, pp. 1426–1433. doi: 10.1109/CVPR.2010.5539801.
- [3] D. Moran, H. Koslowsky, Y. Kasten, H. Maron, M. Galun, and R. Basri, “Deep Permutation Equivariant Structure from Motion.” [Online]. Available: <https://arxiv.org/abs/2104.06703>
- [4] L. Brynte, J. P. Iglesias, C. Olsson, and F. Kahl, “Learning Structure-from-Motion with Graph Attention Networks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2024, pp. 4808–4817.
- [5] F. Khatib, Y. Kasten, D. Moran, M. Galun, and R. Basri, “RESfM: Robust Deep Equivariant Structure from Motion.” [Online]. Available: <https://arxiv.org/abs/2404.14280>
- [6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [8] D. Q. Huynh, “Metrics for 3D Rotations: Comparison and Analysis,” *Journal of Mathematical Imaging and Vision*, vol. 35, no. 2, pp. 155–164, Oct. 2009, doi: 10.1007/s10851-009-0161-2.
- [9] A. R. Geist, J. Frey, M. Zhobro, A. Levina, and G. Martius, “Learning with 3D rotations, a hitchhiker’s guide to $SO(3)$,” in *Proceedings of the 41st International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 15331–15350.

- [10] R. Brégier, “Deep Regression on Manifolds: A 3D Rotation Case Study.” [Online]. Available: <https://arxiv.org/abs/2103.16317>
- [11] A. Kendall and R. Cipolla, “Geometric Loss Functions for Camera Pose Regression with Deep Learning.” [Online]. Available: <https://arxiv.org/abs/1704.00390>
- [12] J. Tung *et al.*, “MegaScenes: Scene-Level View Synthesis at Scale,” in *ECCV*, 2024.
- [13] J. Wang, N. Karaev, C. Rupprecht, and D. Novotny, “VGGsFM: Visual Geometry Grounded Deep Structure From Motion,” 2023.
- [14] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, “VGGT: Visual Geometry Grounded Transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [15] Y. Wang, Z. Chen, and X. X. Zhu, “Learning Generalizable Shape Completion with SIM(3) Equivariance.” [Online]. Available: <https://arxiv.org/abs/2509.26631>